look at authentication for web API applications, used by client-side and mobile apps and at how the authentication system changes for those scenarios.

Another thing to consider is where you store the authentication details for users of your app. In figure 23.2 I showed the authentication services loading the user authentication details from your app's database, but that's only one option.

Another option is to delegate the authentication responsibilities to a third-party identity provider, such as Okta, Auth0, Azure Active Directory B2B/B2C, or even Facebook. These manage users for you, so user information and passwords are stored in their database rather than your own. The biggest advantage of this approach is that you don't have to worry about making sure your customer data is safe; you can be pretty sure that a third party will protect it, as it's their whole business.

TIP Wherever possible, I recommend this approach, as it delegates security responsibilities to someone else. You can't lose your users' details if you never had them! Make sure to understand the differences in providers, however. With a provider like Auth0, *you* would own the profiles created, whereas with a provider like Facebook, you don't!

Each provider has instructions on how to integrate with their identity services, ideally using the OpenID Connect (OIDC) specification. This typically involves configuring some authentication services in your application, adding some configuration, and delegating the authentication process itself to the external provider. These providers can be used with your API apps too, as I discuss in chapter 25.

NOTE Hooking up your apps and APIs to use an identity provider can require a fair amount of tedious configuration, both in the app and the identity provider, but if you follow the provider's documentation you should have plain sailing. For example, you can follow the documentation for adding authentication to a traditional web app using Microsoft's Identity Platform here: http://mng.bz/4D9w.